

Mock Code Challenge - Game Tracker (Object Relationships)

- Due No Due Date
- Points 1
- Submitting a website url

 <https://github.com/learn-co-curriculum/python-p3-mock-challenge-game-tracker>  <https://github.com/learn-co-curriculum/python-p3-mock-challenge-game-tracker/issues/new>

For this assignment, we'll be working with a game tracking domain.

We have three models: `Game` , `Player` , and `Result` .

For our purposes, a `Game` has many `Result` s, a `Player` has many `Result` s, and a `Result` belongs to a `Player` and to a `Game` .

`Game` - `Player` is a many to many relationship.

Note: You should draw your domain on paper or on a whiteboard *before you start coding*. Remember to identify a single source of truth for your data.

Topics

- Classes and Instances
- Class and Instance Methods
- Variable Scope
- Object Relationships
- lists and list Methods

Instructions

To get started, run `pipenv install` while inside of this directory. Then run `pipenv shell` to jump into the shell.

Build out all of the methods listed in the deliverables. The methods are listed in a suggested order, but you can feel free to tackle the ones you think are easiest. Be careful: some of the later methods rely on earlier ones.

Remember! This code challenge has tests to help you check your work. You can run `pytest` to make sure your code is functional before submitting.

We've provided you with a tool that you can use to test your code. To use it, run `python debug.py` from the command line. This will start a `ipdb` session with your classes defined. You can test out the methods that you write here. You can add code to the `debug.py` file to define variables and create sample instances of your objects.

Writing error-free code is more important than completing all of the deliverables listed - prioritize writing methods that work over writing more methods that don't work. You should test your code in the console as you write.

Similarly, messy code that works is better than clean code that doesn't. First, prioritize getting things working. Then, if there is time at the end, refactor your code to adhere to best practices. When you encounter duplicated logic, extract it into a shared helper method.

Before you submit! Save and run your code to verify that it works as you expect. If you have any methods that are not working yet, feel free to leave comments describing your progress.

Deliverables

Write the following methods in the classes in the files provided. Feel free to build out any helper methods if needed.

Initializers and Properties

Game

- `Game __init__(self, title)`
 - `Game` is initialized with a title
- `Game property title`
 - Returns the game's title
 - Titles must be of type `str`
 - Titles must be longer than 0 characters
 - Should **not be able** to change after the game is instantiated
 - *hint: `hasattr()`*

Player

- `Player __init__(self, username)`
 - `Player` is initialized with a username
- `Player` property `username`
 - Returns the player's username
 - Usernames must be of type `str`
 - Usernames must be between 2 and 16 characters, inclusive.
 - Should **be able** to change after the player is instantiated

Result

- `Result __init__(self, player, game, score)`
 - `Result` is initialized with a `Player` instance, a `Game` instance, and a score.
- `Result` property `score`
 - Returns the result's score
 - Scores must be of type `int`
 - Scores must be between 1 and 5000, inclusive
 - Should **not be able** to change after the result is instantiated
 - *hint: `hasattr()`*

Object Relationship Methods and Properties

Result

- `Result` property `player`
 - Returns the player object for that result
 - Must be of type `Player`
- `Result` property `game`
 - Returns the game object for that result
 - Must be of type `Game`

Player

- `Player results()`
 - Returns a list of all results for that player
 - Results must be of type `Result`
- `Player games_played()`
 - Returns a **unique** list of all games played by a particular player
 - Games must be of type `Game`

Game

- `Game results()`
 - Returns a list of all results for that game
 - Results must be of type `Result`
- `Game players()`
 - Returns a **unique** list of all players that played a particular game
 - Players must be of type `Player`

Aggregate and Association Methods

Player

- `Player played_game(game)`
 - Receives a **game object** as argument
 - Returns `True` if the player has played the game object provided
 - Returns `False` otherwise
- `Player num_times_played(game)`
 - Receives a **game object** as argument
 - Returns the number of times the player has played the game instance provided
 - Returns 0 if the player never played the game provided

Game

- `Game average_score(player)`
 - Receives a **player object** as argument
 - Returns the average of all the player's scores for a particular game instance
 - Reminder: you can calculate the average by adding up all the results' scores of the player specified and dividing by the number of those results

Bonus: Aggregate and Association Method

- `Player classmethod highest_scored(game)`
 - Receives a **game object** as argument
 - Returns the `Player` instance with the highest average score for the game provided.
 - Returns `None` if there are no players that played the game provided.
 - *hint: will need a way to remember all player objects*
 - *hint: do you have a method to get the average score on a game for a particular player?*
 - Uncomment lines 151-161 in the `player_test` file

Bonus: For any invalid inputs raise an **Exception** .

- First, **comment out** the following lines
 - `game_test.py`
 - lines 25-26
 - `player_test.py`
 - lines 25-26, 40-41, 44-45
 - `result_test.py`
 - lines 29-31
- Then, **uncomment** the following lines in the test files
 - `game_test.py`
 - lines 29-30, 40-41
 - `player_test.py`
 - lines 29-30, 48-49, and 52-53

- **result_test.py**
- lines 34-35, 38-39, 50-51, and 54-55